

1 . F O U N D A T I O N S O F C N N S

A N T O N I O D Í A Z - C A N O

A J D I A Z C A N O @ C O M I L L A S . E D U

INDEX

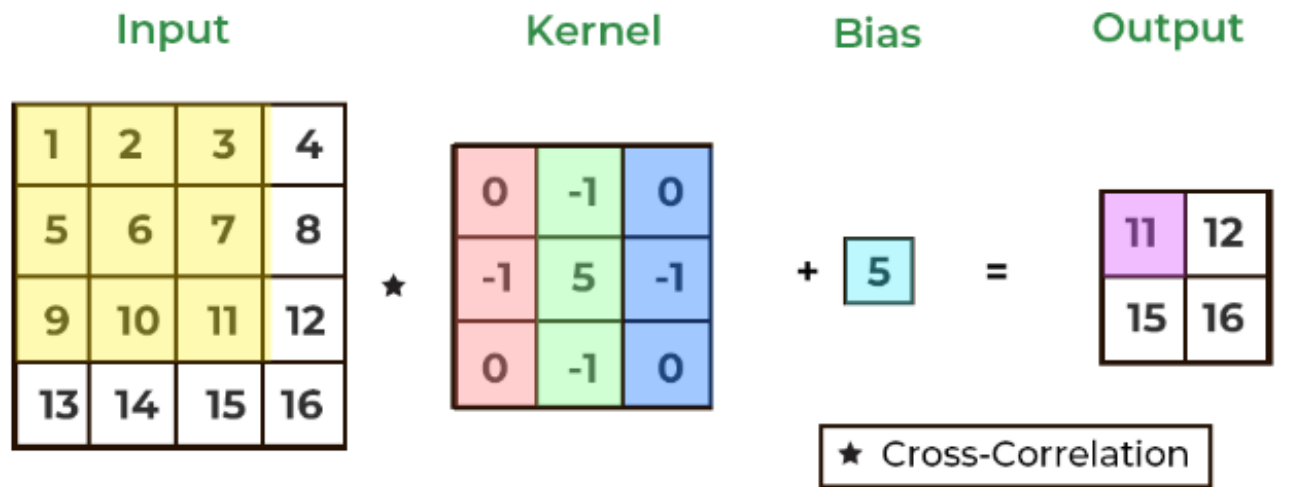
1. Computer Vision Concepts
2. Computer Vision Architectures
3. Transfer Learning



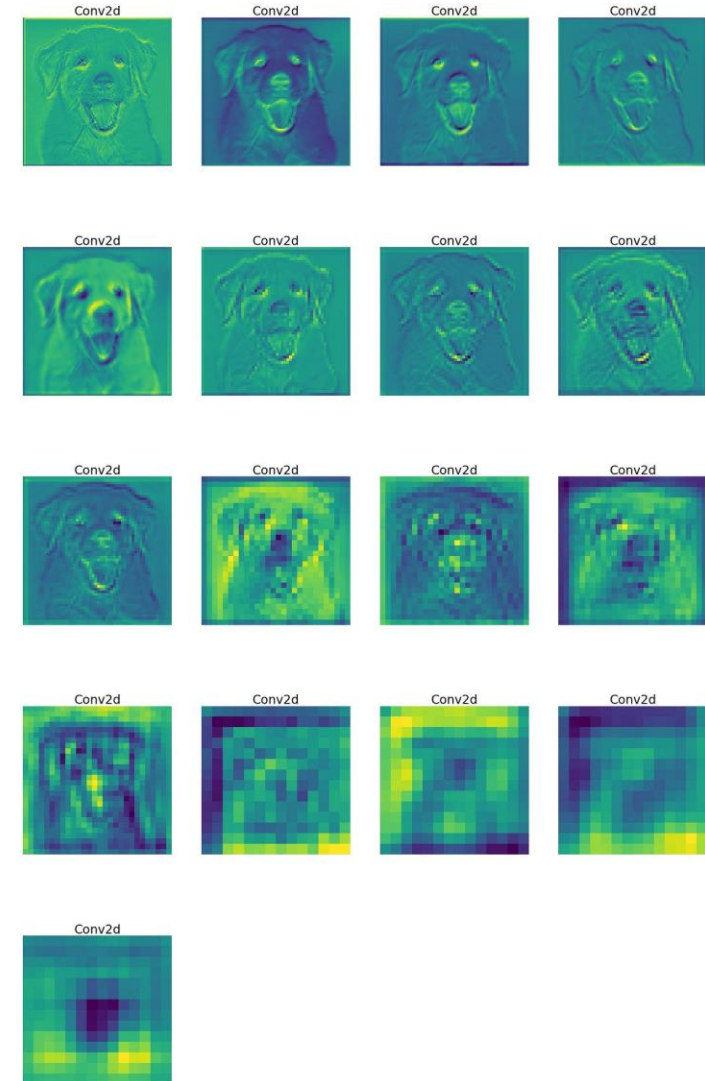
1 . C O M P U T E R V I S I O N C O N C E P T S

REVIEW OF CNNs

What is a convolution? Why is it useful?



$$1 \times 0 + 5 \times -1 + 9 \times 0 + 2 \times -1 + 6 \times 5 + 10 \times -1 + 3 \times 0 + 7 \times -1 + 11 \times 0 + 5 = 11$$



REVIEW OF CNNs

Padding

0	0	0	0	0	0	0	0
0	3	3	4	4	7	0	0
0	9	7	6	5	8	2	0
0	6	5	5	6	9	2	0
0	7	1	3	2	7	8	0
0	0	3	7	1	8	3	0
0	4	0	4	3	2	2	0
0	0	0	0	0	0	0	0

$6 \times 6 \rightarrow 8 \times 8$

*

1	0	-1
1	0	-1
1	0	-1

3×3

=

-10	-13	1			
-9	3	0			

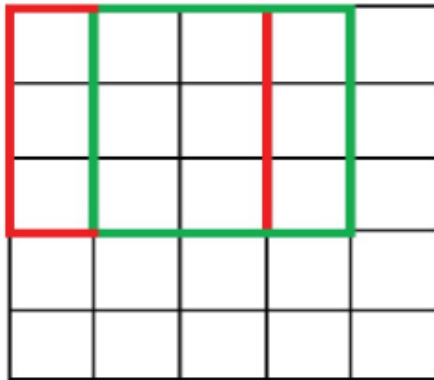
6×6



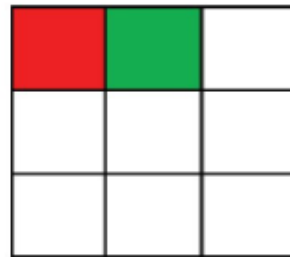
REVIEW OF CNNs

Stride

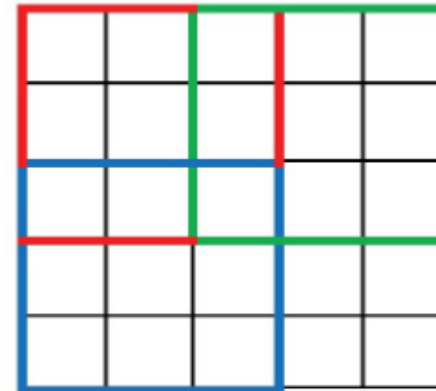
Convolution
with Stride=1



Output



Convolution
with Stride=2

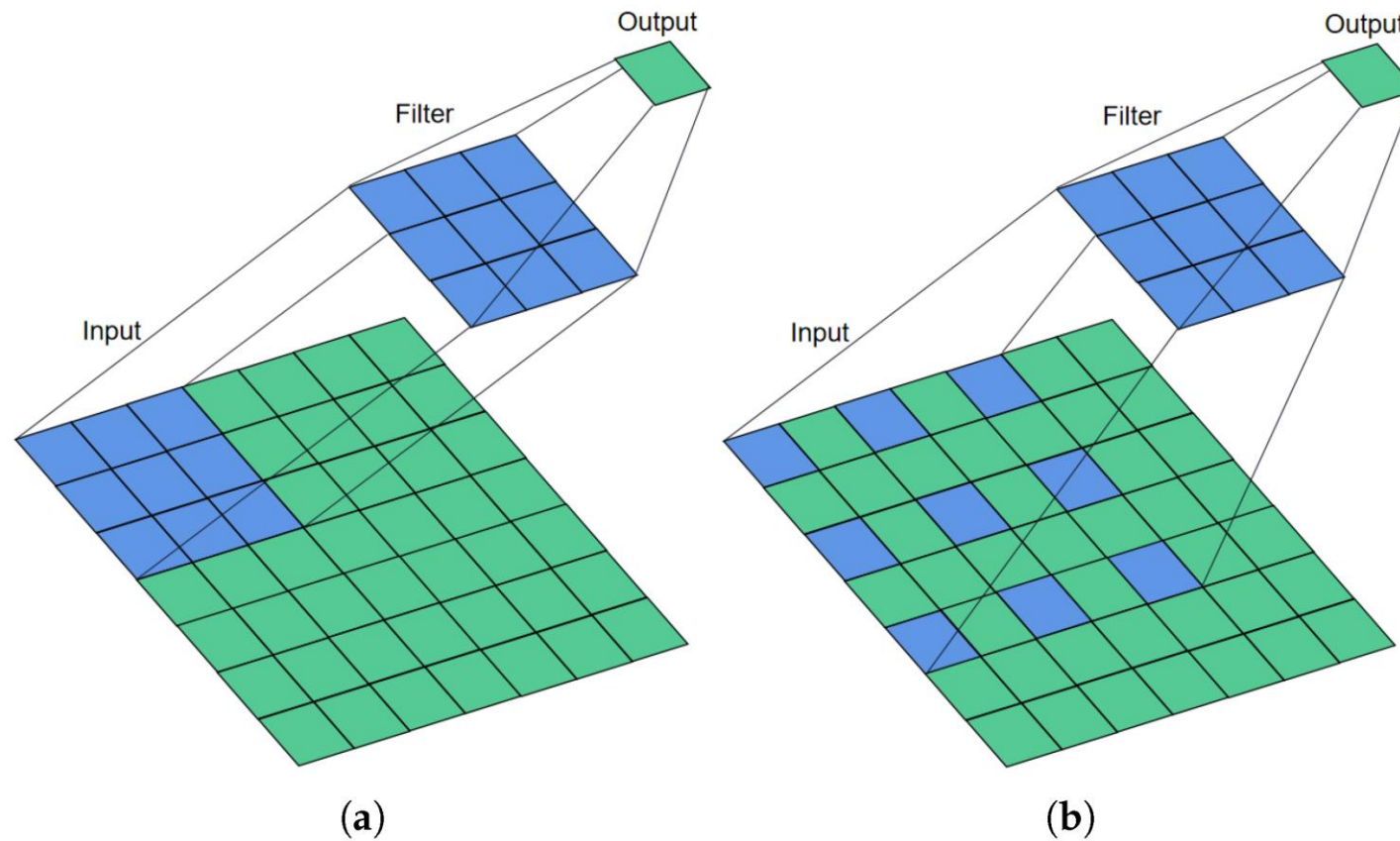


Output



REVIEW OF CNNs

Dilation

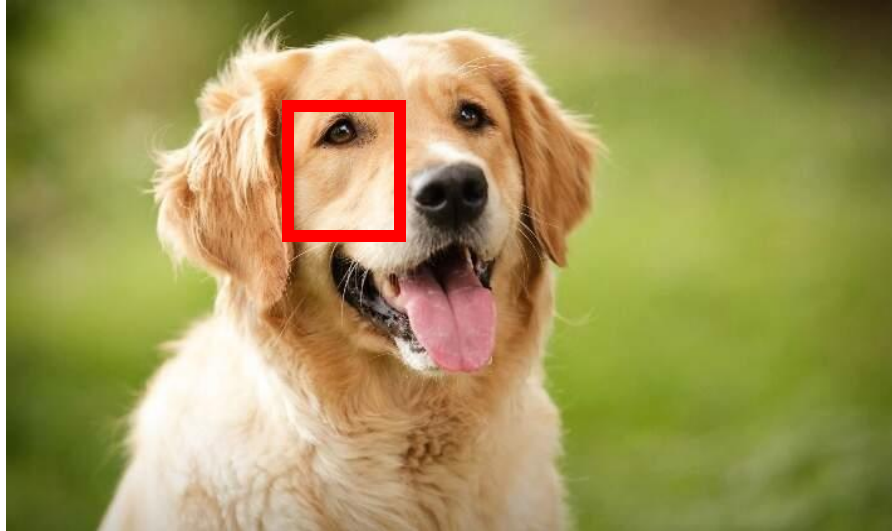


REVIEW OF CNNs

Parameter Sharing

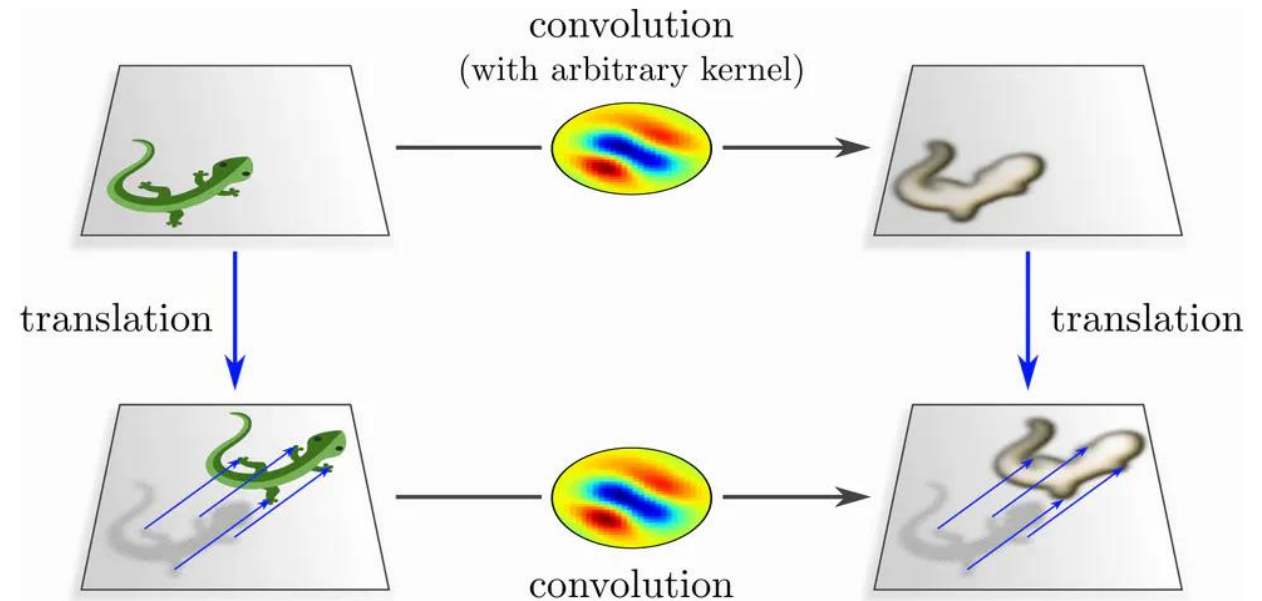
Same kernel weights are applied at every spatial location.

Local Connectivity



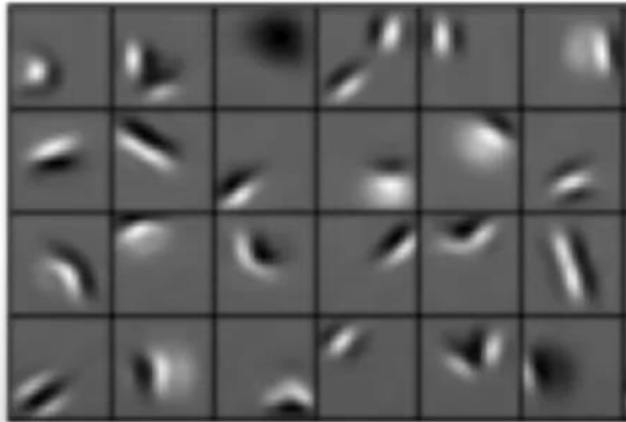
Translation equivariance

$$\text{conv}(T(X)) = T(\text{conv}(X))$$



FEATURES ALONG THE NETWORK

Low level features



Edges, dark spots

Mid level features



Eyes, ears, nose

High level features



Facial structure



RECEPTIVE FIELD

$$RF_L = RF_{L-1} + (k_L - 1)$$

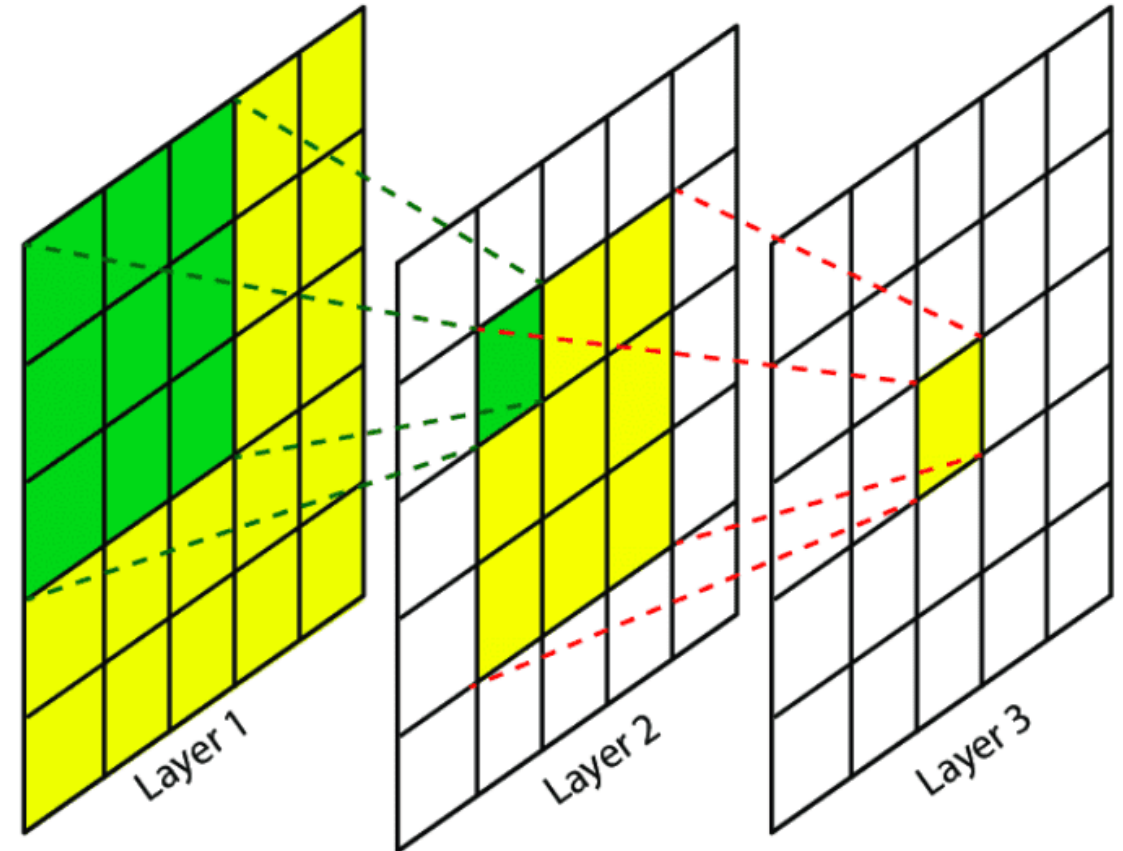
$$RF_1 = 1$$

$$RF_2 = RF_1 + (3 - 1) = 3$$

$$RF_3 = RF_2 + (3 - 1) = 5$$

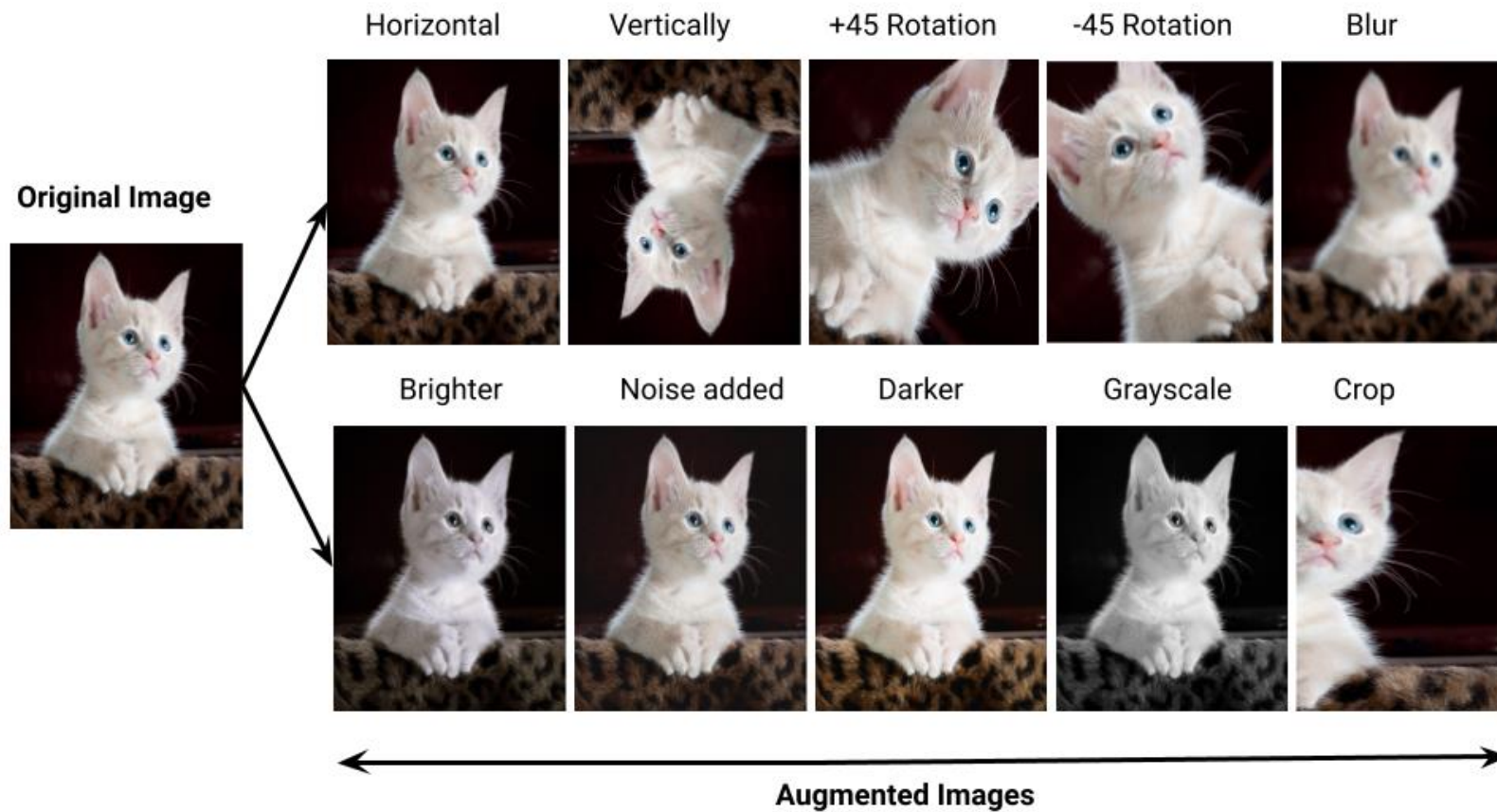
- Layer 3 "sees" a 3x3 region of layer 2.
- Layer 2 "sees" a 3x3 region of layer 1.
- Layer 3 "sees" a 5x5 region of layer 1.

This explains the idea of the previous slide!



DATA AUGMENTATION

Different input, but same label! It reduces overfitting and generalizes more.

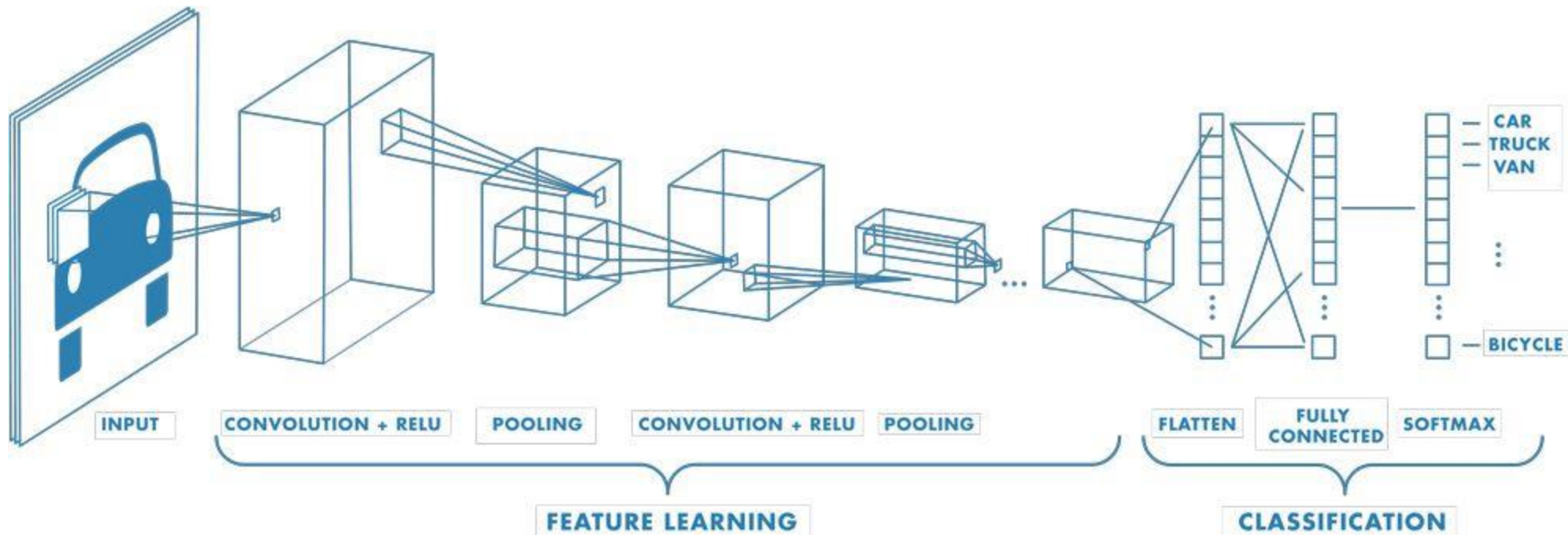


2. COMPUTER VISION ARCHITECTURES

STANDARD CNN BASELINE

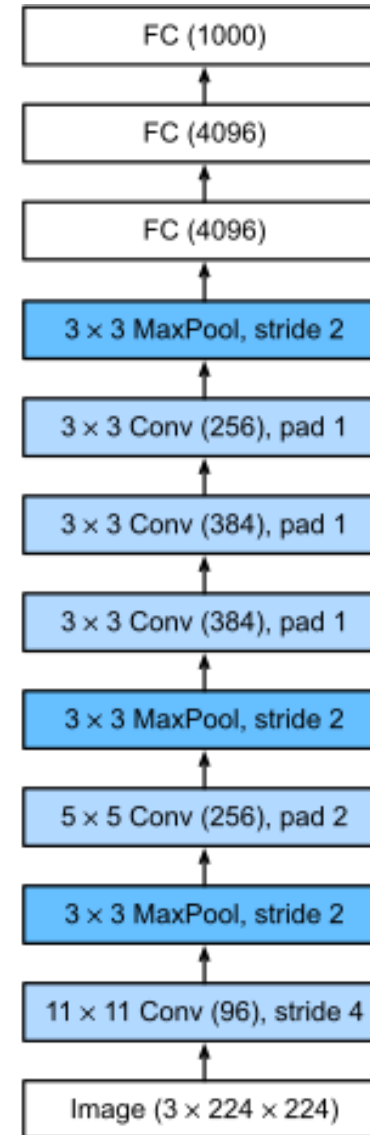
Simple and intuitive, but suffers from parameter explosion when increasing depth!

CNN/ReLU/Pooling --> Flatten --> MLP



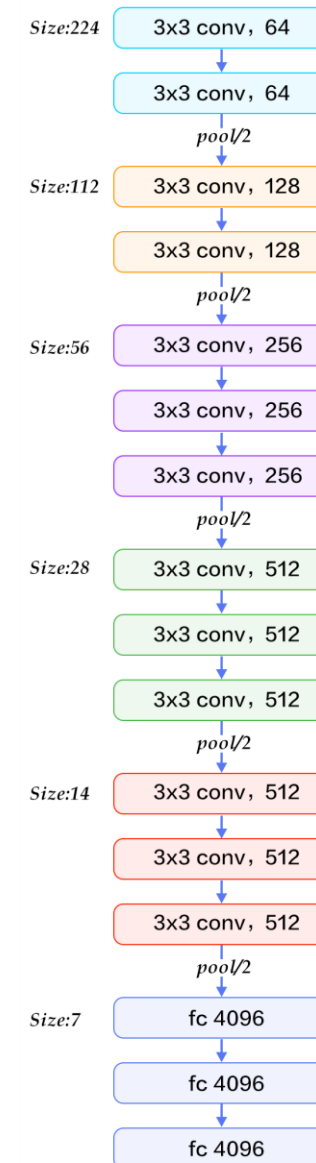
ALEXNET

- ReLU for faster convergence
- Dropout to reduce overfitting



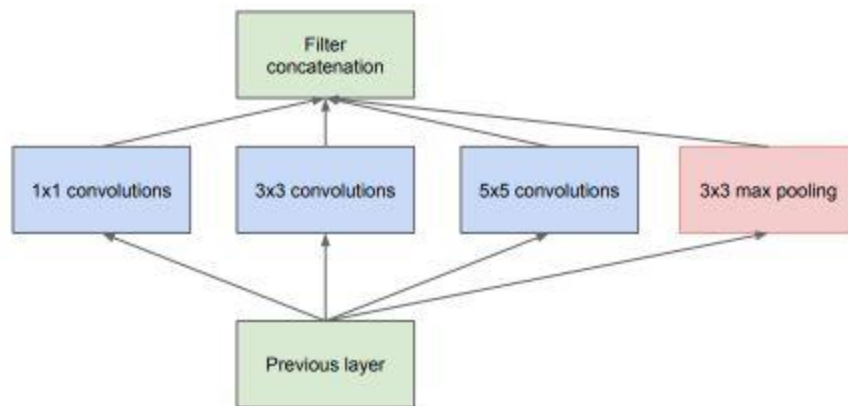
VGGNET

- Small 3x3 convolution filters
- Deep architecture (19 layers in VGG-19)

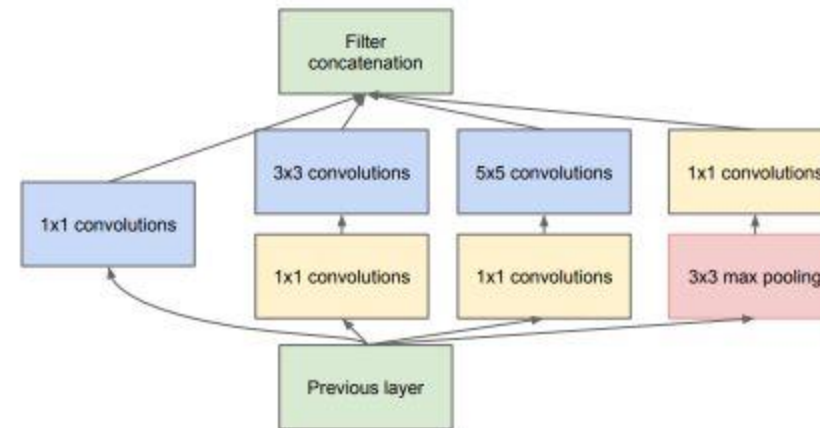


GOOGLNET

- 1x1 convolution for dimensionality reduction
- Inception module
- Auxiliary classifiers
- Parameter efficient



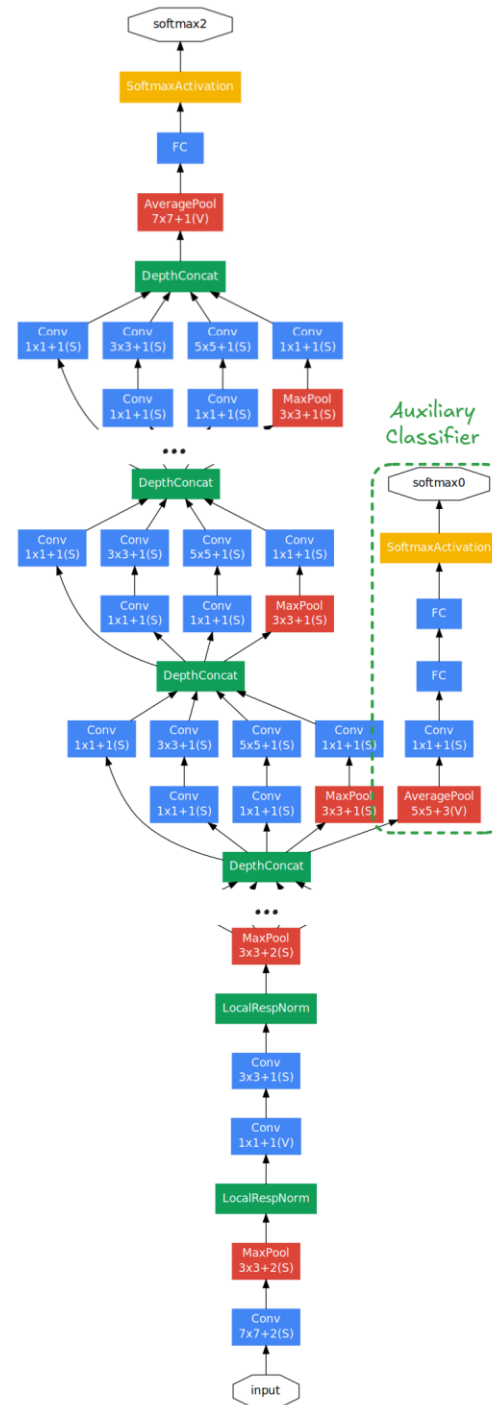
(a) Inception module, naïve version



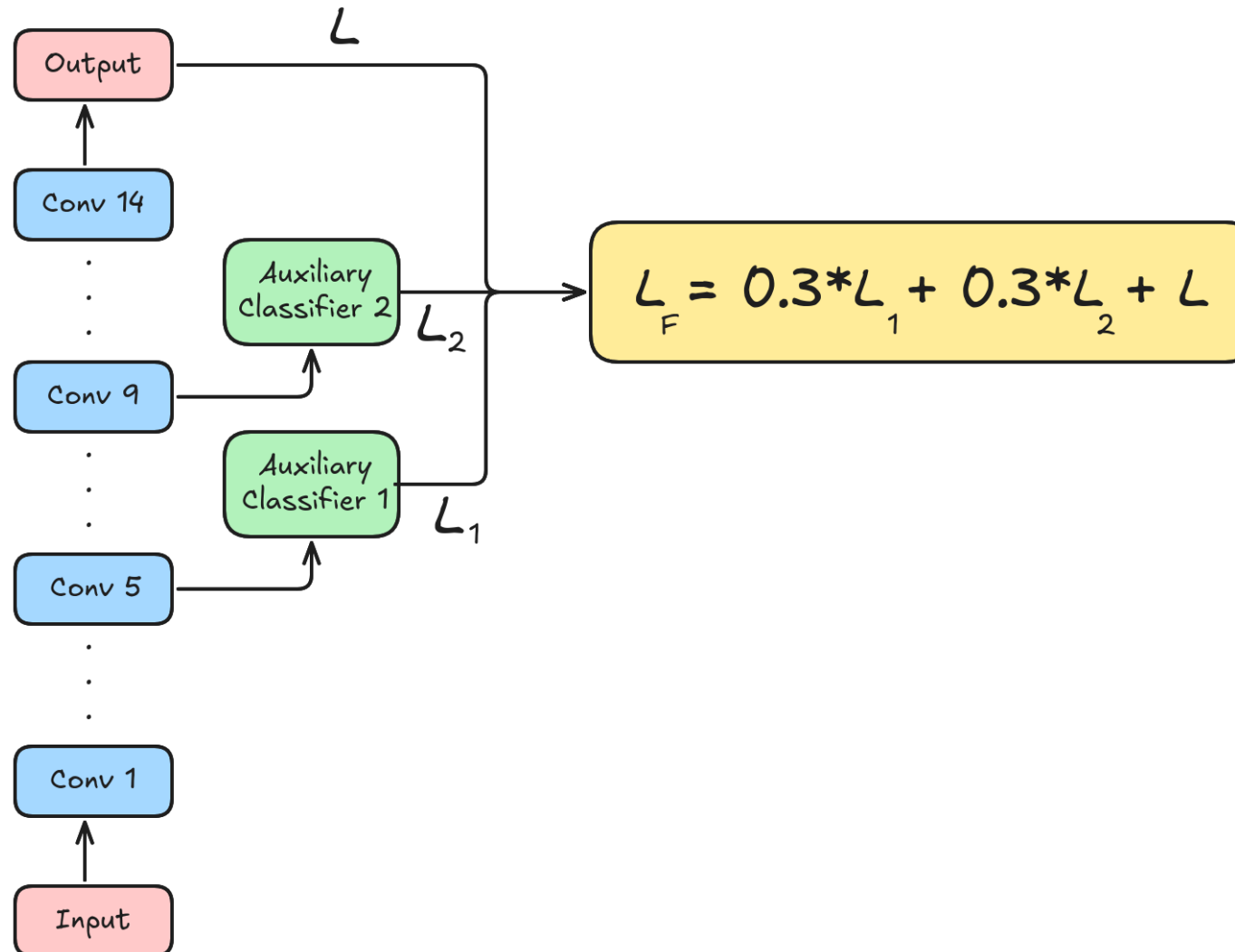
(b) Inception module with dimension reductions



GOOGLNET

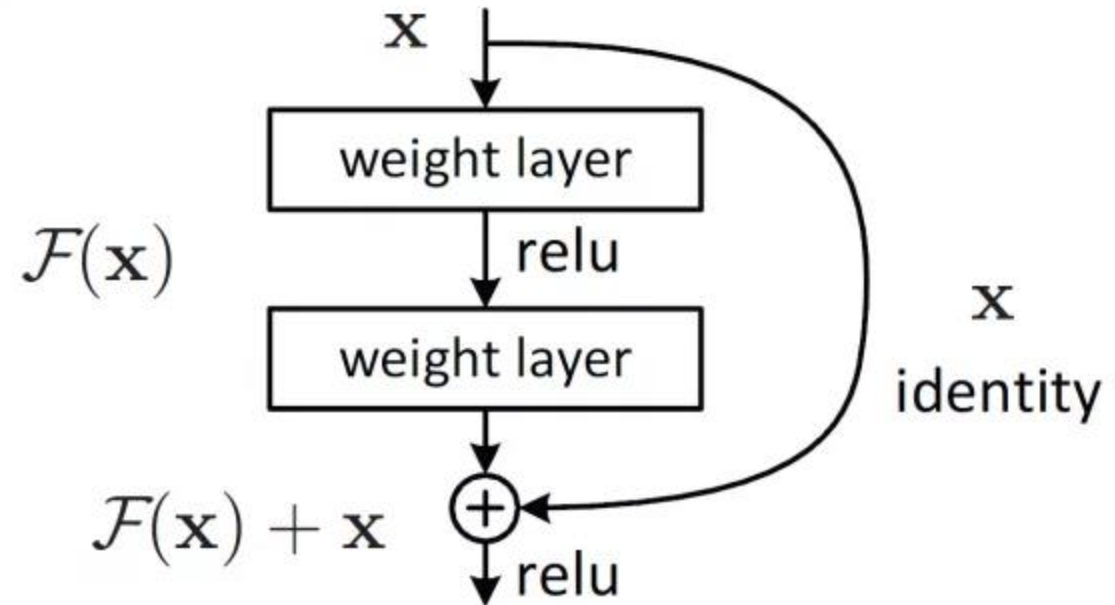


GOOGLNET



RESNET

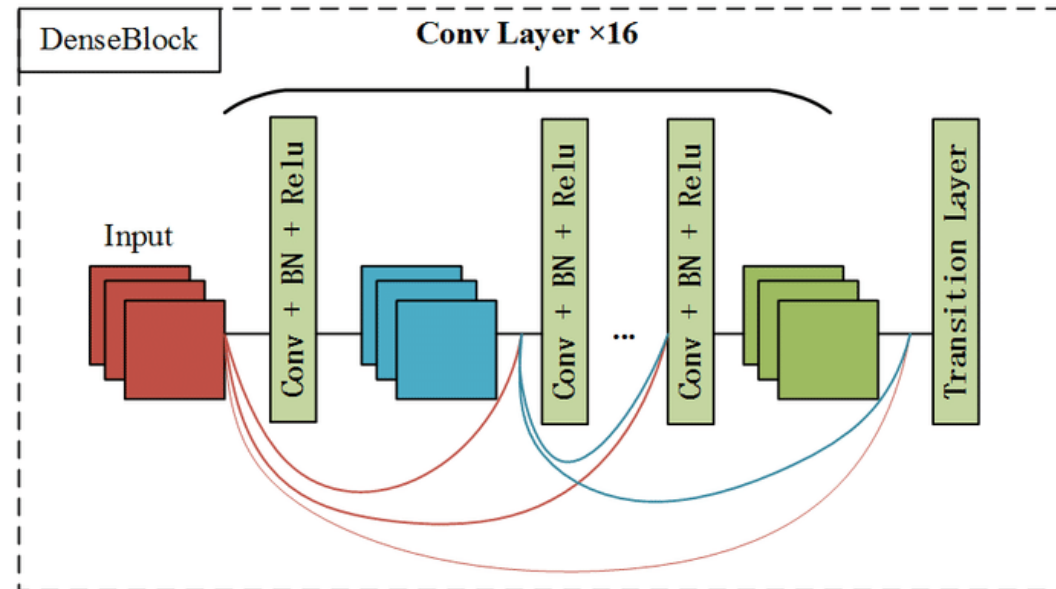
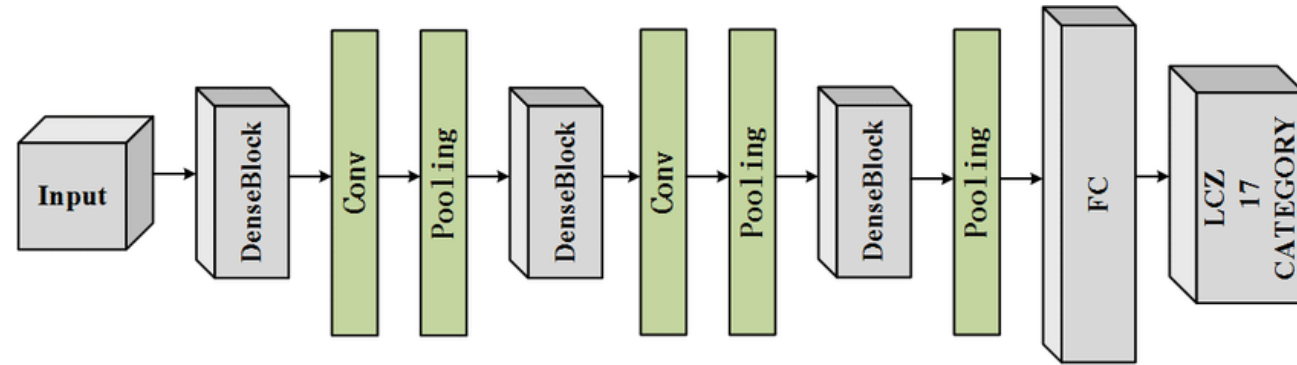
- Residual connections
- Very deep architectures!



$$\frac{\partial \mathcal{L}}{\partial x} = \frac{\partial \mathcal{L}}{\partial y} \frac{\partial y}{\partial x} = \frac{\partial \mathcal{L}}{\partial y} \left(\frac{\partial \mathcal{F}(x) + x}{\partial x} \right) = \frac{\partial \mathcal{L}}{\partial y} \left(\frac{\mathcal{F}(x)}{\partial x} + 1 \right) = \frac{\partial \mathcal{L}}{\partial y} + \frac{\partial \mathcal{L}}{\partial y} \frac{\partial \mathcal{F}(x)}{\partial x}$$

DENSENET

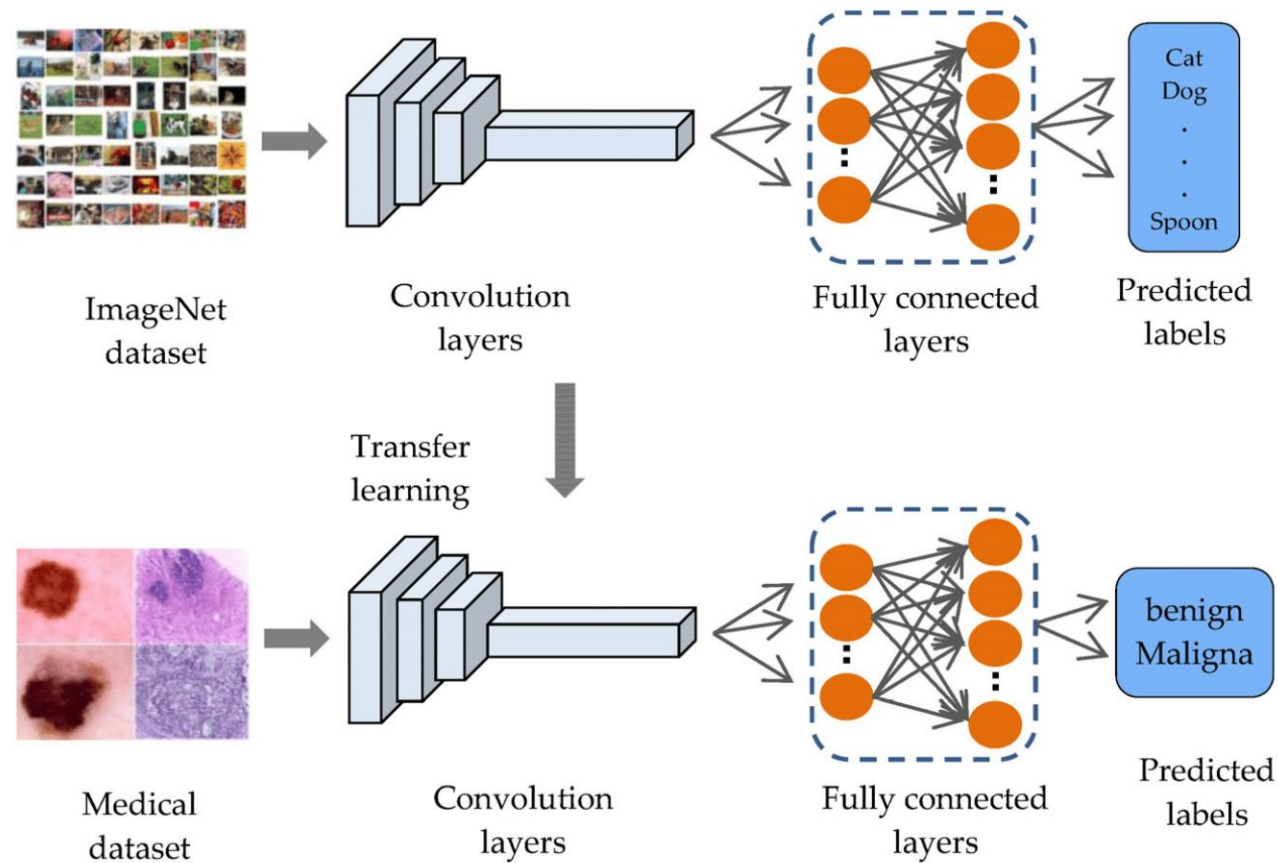
- Dense connectivity
- Parameter efficient



6 . T R A N S F E R L E A R N I N G

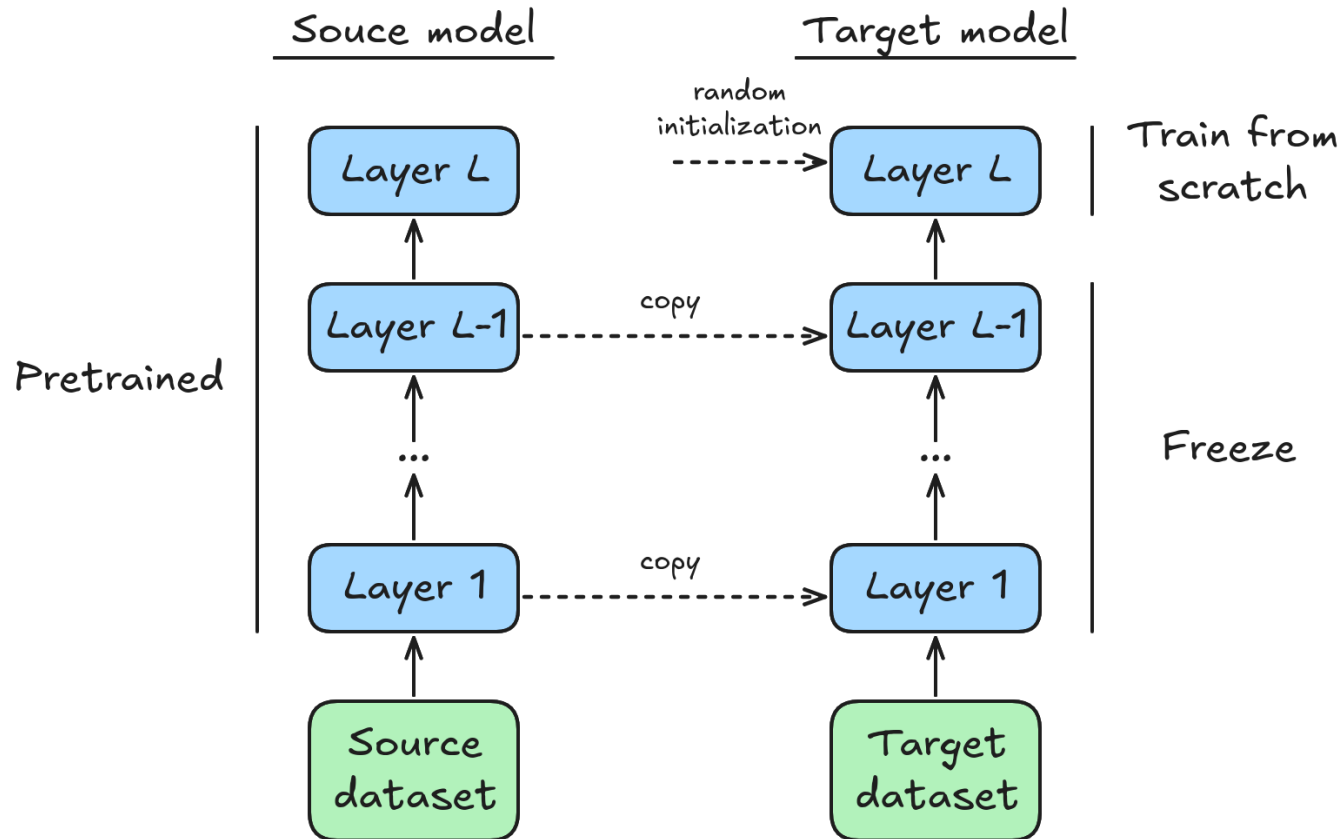
TRANSFERRING KNOWLEDGE

Idea: Start with an already trained model instead of from scratch!

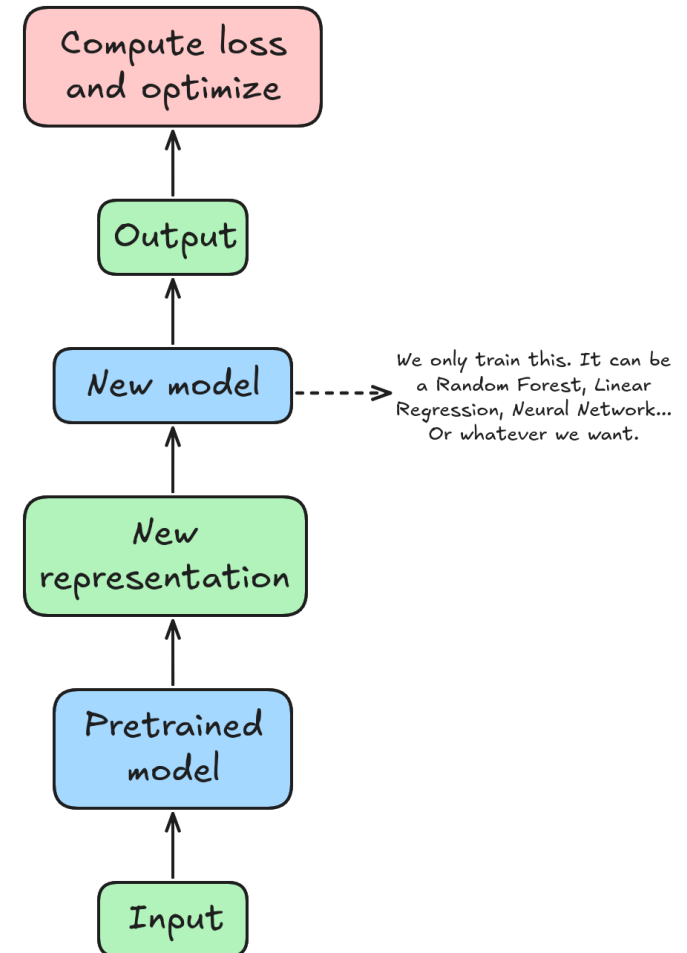


FINE-TUNING

Reuse architecture



Feature extractor



FINE-TUNING

	Similar dataset	Different dataset
A lot of data	Fine-tune some or all model layers	Either fine-tune all model layers or train from scratch!
Little data	Use linear classifier on final layer or use the backbone as feature extractor	(Bad scenario) Extract features from intermediate layers or try another model/strategy



T H A N K S !

